SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I RAHIMA BANU N-192525009 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:RAHIMA BANU N

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

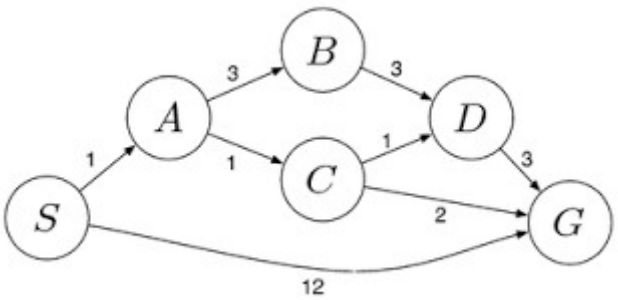
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

QUESTION 1: WATER JUG PROBLEM

Problem:
There are two jugs of capacity 4 gallons and 3 gallons, without measuring marks. Using a water pump and the ability to pour water between jugs or onto the ground, obtain exactly 2 gallons in the 4-gallon jug.

Solution:

Let Jug A have a capacity of 4 gallons and Jug B have a capacity of 3 gallons.

The initial state is (0, 0), where the first value represents water in Jug A and the second value represents water in Jug B.

Steps to solve the problem:

Step 1: Initially, both jugs are empty. State: (0, 0).

Step 2: Fill Jug B completely. State: (0, 3).

Step 3: Pour water from Jug B into Jug A. State: (3, 0).

Step 4: Fill Jug B completely again. State: (3, 3).

Step 5: Pour water from Jug B into Jug A until Jug A is full. State: (4, 2).

Step 6: Empty Jug A onto the ground. State: (0, 2).

Step 7: Pour the remaining 2 gallons from Jug B into Jug A. State: (2, 0).

Final Answer:

Exactly 2 gallons of water are obtained in the 4-gallon jug.

Conclusion:

The water jug problem is solved by applying a sequence of valid actions that transforms the initial state (0, 0) into the goal state (2, 0).

QUESTION 2: MARS ROVER AS AN INTELLIGENT AGENT

A Mars Rover is an intelligent agent designed to explore the surface of Mars, collect scientific samples, analyse the environment, and transmit information to Earth.

(i) What are the percepts for this agent?

Percepts are the information received by an intelligent agent from its environment through sensors.

The Mars Rover receives the following percepts:

- Images of the Martian surface from cameras.
- Distance and obstacle information from sensors.
- Temperature, pressure, and radiation measurements.
- Chemical composition of soil and rock samples.
- Terrain conditions such as slopes, rocks, and craters.
- Battery level and equipment status.
- Location and orientation information.

(ii) Characterize the operating environment.

The Mars Rover operates in the following environment:

Partially observable: The rover cannot observe the entire Martian environment at once.

Single-agent: It primarily performs its mission independently, although it communicates with Earth-based operators.

Stochastic: Outcomes may be uncertain because of terrain conditions, equipment limitations, and communication delays.

Sequential: Each action affects future actions and the progress of the mission.

Dynamic: Environmental conditions may change, and the rover's surroundings may present new challenges.

Continuous: Position, distance, temperature, and movement involve continuous values.

Partially known: The rover has some prior information about Mars, but many environmental details remain unknown.

(iii) What are the actions the agent can take?

- The Mars Rover can perform the following actions:
- Move forward, backward, and turn.
- Capture images of the Martian surface.
- Avoid obstacles and select safe routes.
- Collect soil and rock samples.
- Analyse samples using scientific instruments.
- Measure environmental conditions.
- Send collected data and images to Earth.
- Manage battery power and enter a safe mode when required.

(iv) How can one evaluate the performance of the agent?

The performance of the Mars Rover can be evaluated using the following criteria:

- Number of useful scientific samples collected and analysed.
- Accuracy and quality of scientific data.
- Distance explored and area covered.
- Ability to avoid obstacles and prevent damage.

- Efficient use of battery power.
- Successful transmission of data to Earth.
- Completion of mission objectives within the available time and resources.
- A rover performs well when it achieves its scientific objectives safely, accurately, and efficiently.

(v) What sort of agent architecture is most suitable? Why?

A model-based, goal-based agent with utility-based decision-making is most suitable for the Mars Rover.

Reasons:

A model-based agent maintains an internal representation of the environment because the entire Martian surface cannot be observed at once.

A goal-based agent chooses actions that help it achieve goals, such as reaching a location or collecting samples.

A utility-based agent compares alternatives and selects actions that balance safety, scientific value, energy consumption, and time.

A learning component can improve decisions using experience and newly collected data.

Conclusion:

A hybrid intelligent-agent architecture is suitable because the rover must understand its environment, plan actions, achieve scientific goals, and make safe decisions under uncertainty.

QUESTION 3: EIGHT-QUEENS PROBLEM USING SEARCH STRATEGIES

Problem:

Place 8 queens on an 8 × 8 chessboard such that no queen attacks any other queen. Queens must not share the same row, column, or diagonal.

Problem Formulation:

Initial State:

An empty 8 × 8 chessboard.

State Space:

All possible configurations of queens on the chessboard.

Actions:

Place a queen in a safe square in the next column.

Transition Model:

The action places one queen in the selected square and generates a new board configuration.

Goal Test:

Check whether all 8 queens are placed without any two queens attacking each other.

Path Cost:

Each queen placement can be assigned a cost of 1. The total cost of reaching a solution is 8.

Constraints:

No two queens can occupy the same row.

No two queens can occupy the same column.

No two queens can occupy the same diagonal.

If a queen is placed at position (r_i, c_i) and another at position (r_j, c_j), they must satisfy the following conditions:

$r_i \neq r_j$

$|r_i - r_j| \neq |c_i - c_j|$

The first condition prevents queens from sharing a row, while the second prevents diagonal attacks. Placing one queen per column automatically prevents column conflicts.

Search Strategy:

Backtracking Search, which uses Depth-First Search (DFS), is an effective method for solving the Eight-Queens problem.

Steps:

Step 1: Start with an empty chessboard.

Step 2: Place a queen in a safe position in the first column.

Step 3: Move to the next column and place another queen in a safe position.

Step 4: Continue until all 8 queens are placed.

Step 5: If no safe position exists in a column, remove the previously placed queen and try another position.

Step 6: Repeat until a valid arrangement is found.

Pseudocode:

N_QUEENS(row):

```
IF row == 9:
    RETURN SUCCESS

FOR col = 1 TO 8:
    IF SAFE(row, col):
        PLACE_QUEEN(row, col)

        IF N_QUEENS(row + 1) == SUCCESS:
            RETURN SUCCESS

        REMOVE_QUEEN(row, col)

RETURN FAILURE
```

The SAFE(row, col) function checks whether placing a queen at that position causes a row, column, or diagonal conflict with an existing queen. The algorithm starts with N_QUEENS(1).

Example of a Valid Solution:

One valid arrangement is represented by the column positions of queens in rows 1 through 8:

[1, 5, 8, 6, 3, 7, 2, 4]

This means:

Row 1: Column 1

Row 2: Column 5

Row 3: Column 8

Row 4: Column 6

Row 5: Column 3

Row 6: Column 7

Row 7: Column 2

Row 8: Column 4

No two queens share a row, column, or diagonal.

Conclusion:

Backtracking search systematically explores possible configurations and reverses incorrect choices until it finds a valid solution.

QUESTION 4: OLA CAB BOOKING USING A PROBLEM-SOLVING AGENT

Problem:

A customer uses the OLA Cab booking application to travel from a starting location to a destination. The customer can select different cab types, such as Mini, Micro, Sedan, Shared, or Prime, based on comfort, cost, and travel requirements.

Type of Problem-Solving Agent:

A goal-based problem-solving agent with utility-based decision-making is suitable for this problem.

Explanation:

A goal-based agent works towards reaching the customer's destination.

A utility-based agent compares available cab options based on fare, estimated travel time, comfort, and availability.

The agent selects a suitable cab that satisfies the customer's requirements and provides the best overall choice.

Problem Formulation:

Initial State:

The customer's current location.

Goal State:

The customer reaches the selected destination.

State Space:

Locations, routes, available cabs, and booking states.

Actions:

Enter destination, select cab type, compare fares, book a cab, and travel along the selected route.

Transition Model:

Each action changes the booking or travel state, such as moving from cab selection to booking confirmation.

Path Cost:

Total travel cost, including fare, travel time, and other relevant factors.

Goal Test:

Check whether the customer has reached the destination.

Pseudocode:

OLA_CAB_BOOKING()

```
START = Get_Current_Location()
GOAL = Get_Destination()

IF START == GOAL:
    DISPLAY "Already at destination"
    RETURN

CAB_LIST = Get_Available_Cabs()

IF CAB_LIST is empty:
    DISPLAY "No cabs available"
    RETURN

DISPLAY CAB_LIST

PREFERENCE = Get_Customer_Preference()

BEST_CAB = Select_Cab(
    CAB_LIST,
    PREFERENCE,
    Fare,
    ETA,
    Comfort
)

IF BEST_CAB is not available:
    DISPLAY "Select another cab"
    RETURN

DISPLAY BEST_CAB, Fare, ETA

IF Customer_Confirms_Booking():
    BOOK_CAB(BEST_CAB)
    DISPLAY "Cab booked successfully"

    WHILE Destination_Not_Reached():
        UPDATE_CAB_LOCATION()
        UPDATE_ETA()

    DISPLAY "Destination reached"

ELSE:
    DISPLAY "Booking cancelled"
```

END

Explanation:

Step 1: The application identifies the customer's current location and destination.

Step 2: It retrieves available cab types.

Step 3: It compares options using fare, estimated arrival time (ETA), availability, and customer preferences.

Step 4: It recommends a suitable cab.

Step 5: After customer confirmation, the booking is created.

Step 6: The application tracks the cab and updates the ETA until the customer reaches the destination.

Conclusion:

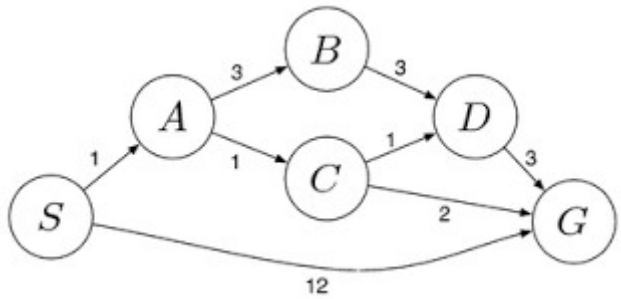
A goal-based agent helps the customer reach the destination, while utility-based decision-making helps select a suitable cab according to cost, comfort, availability, and travel time.

QUESTION 5: UNIFORM COST SEARCH FOR THE DELIVERY NETWORK

Problem:

A logistics company wants to transport packages from starting warehouse S to destination warehouse G at minimum cost. Each edge in the weighted graph represents a route cost.

Given Graph:



The directed edges and their costs are:

S → A = 1

S → G = 12

A → B = 3

A → C = 1

B → D = 3

C → D = 1

C → G = 2

D → G = 3

Uniform Cost Search Algorithm:

Uniform Cost Search expands the node with the lowest cumulative path cost first. It uses a priority queue to maintain the frontier of unexplored paths.

The cumulative cost is calculated as:

g(n) = Sum of all edge costs from S to node n.

Step-by-Step Solution:

Step 1:

Expand S with cost 0.

Successors:

A = 1

G = 12

Priority queue: A(1), G(12).

Step 2:

Expand A with cost 1.

Successors:

B = 1 + 3 = 4

C = 1 + 1 = 2

Priority queue: C(2), B(4), G(12).

Step 3:

Expand C with cost 2.

Successors:

D = 2 + 1 = 3

G = 2 + 2 = 4

Priority queue: D(3), B(4), G(4).

Step 4:

Expand D with cost 3.

Successor:

G = 3 + 3 = 6.

The existing path to G has cost 4, so the more expensive path with cost 6 is ignored.

Priority queue: B(4), G(4).

Step 5:

Expand B with cost 4.

Successor:

D = 4 + 3 = 7.

The existing path to D has cost 3, so the more expensive path with cost 7 is ignored.

Priority queue: G(4).

Step 6:

Expand G with cost 4.

The goal node G is reached, so the search stops.

Comparison of Possible Paths:

Path: S → G

Total cost: 12

Path: S → A → B → D → G

Total cost: 1 + 3 + 3 + 3 = 10

Path: S → A → C → D → G

Total cost: 1 + 1 + 1 + 3 = 6

Path: S → A → C → G

Total cost: 1 + 1 + 2 = 4

Final Answer:

The least-cost path is:

S → A → C → G

Total minimum cost:

Cost = 1 + 1 + 2 = 4

Conclusion:

Uniform Cost Search finds the optimal path from S to G with a minimum total cost of 4. It guarantees an optimal solution when all edge costs are non-negative.